

## Segment-5

### Graphs basic & Traversal Techniques

Graph: A graph is a collection of nodes, called vertices and a collection of line segments, called edges, connecting pairs of vertices.

Graph — Undirected:  
An undirected graph is a graph in which there is no direction associated with the lines

— Directed:  
A directed graph is graph in which each line has a direction associated with it.

Adjacent:  
Two vertices in a graph are said to be adjacent if an edge directly connects them.

Path:  
A path is sequence of vertices in which each vertex is adjacent to the next one.

Cycle:  
A cycle is a path that starts and ends with the same vertex.

## Self edge or loop:

A self ~~loop~~ ~~or~~ edge or loop is an arc that begins and ends at the same vertex.

## • Three types of Directed Graph:

### - Strongly connected graph:

A Directed graph is strongly connected if there is a path from each vertex to every other vertex.

### - Weakly connected graph:

A directed graph is weakly connected if at least 2 vertices are not connected.

### - Disconnected graph:

A graph is disconnected or disjoint if it is not connected.

## Degree:

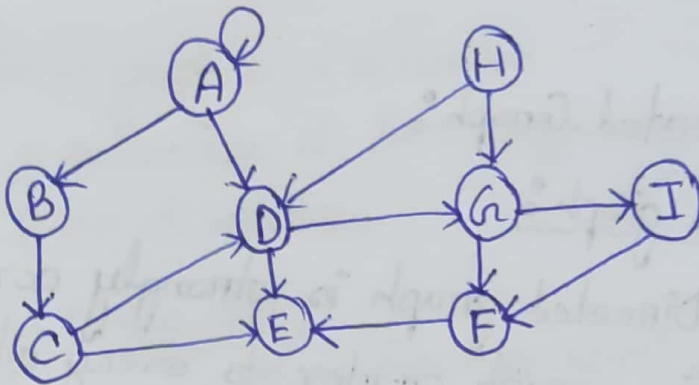
The degree of a vertex is the number of edges incident to it.

Outdegree: The outdegree of a vertex in a digraph is the number of arcs leaving the vertex.

Indegree: The indegree of a vertex in a digraph is the number of arcs entering the vertex.



☐ Show adjacency list and adjacency matrix representation for the graph is given below.

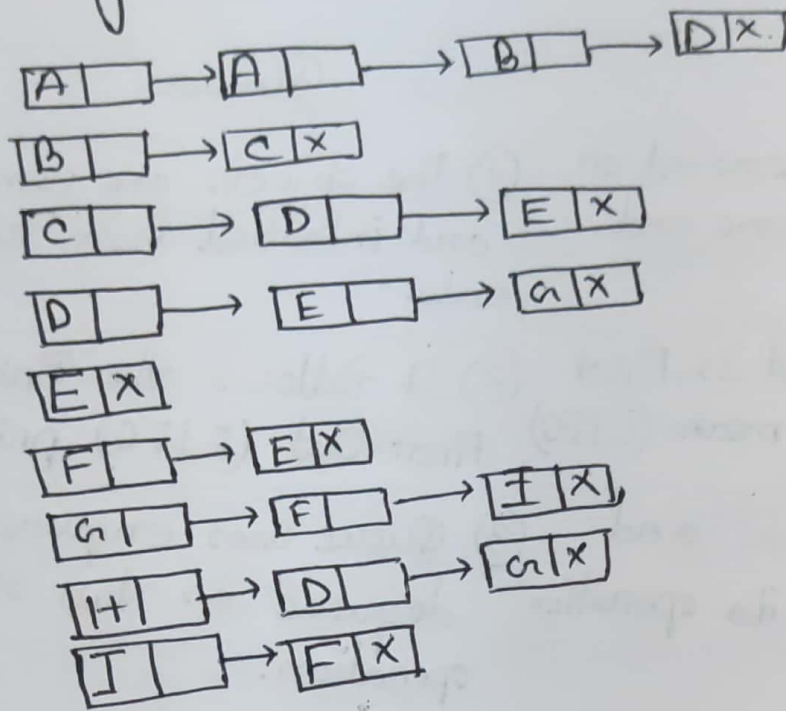


Sol<sup>n</sup>: Adjacency matrix:

|   | A | B | C | D | E | F | G | H | I |
|---|---|---|---|---|---|---|---|---|---|
| A | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| C | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| D | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| F | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| G | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| H | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| I | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |



## Adjacency list:



## Sparse Graph:

It is a graph which has less edge.  
- Adjacency list is more efficient.

## Dense Graph:

It is a graph which has more edge.  
- Adjacency matrix is more efficient.

# Adjacency list vs Adjacency matrix



## Differences Between Queue and Stack:

| Stack                                                                     | Queue                                                                         |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| ① The objects are removed or inserted at the same end.                    | ① The objects are removed and inserted from two different ends.               |
| ② It follows the Last In, First Out principle, <del>it means</del> (LIFO) | ② It follows the First In, First Out (FIFO) principle.                        |
| ③ Stack uses 'push' and 'pop' as two of its operation                     | ③ Queue uses enqueue and dequeue as two of its operation.                     |
| ④ Insertion and deletion of elements take place from the 'Top'.           | ④ Insertion uses the rear end, and deletion uses the front end.               |
| ⑤ When $Top == max - 1$ , it means that the stack is full.                | ⑤ When $rear == max - 1$ , it means that the queue is full                    |
| ⑥ Stack does not have any types.                                          | ⑥ Queue has three types - circular queue, priority queue, double-ended queue. |
| ⑦ The implementation is simpler in stack                                  | ⑦ The implementation is comparatively more complex in a Queue.                |

## Traverse Graph

There are 2 standard graph traversals:

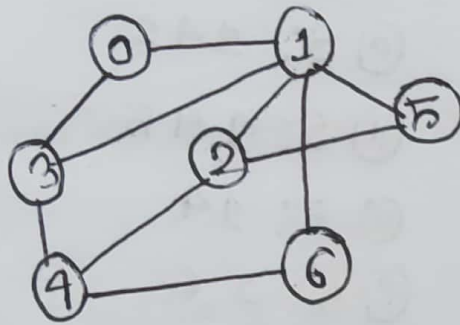
- Depth first
- Breadth first

### Breadth-First Traversal (BFS):

In the BFS of a graph, we process all adjacent vertices of a vertex before going to the next level.

- The BFS uses queue.

#### Example:



#### adjacency list:

0: 1, 3  
1: 0, 2, 3, 5, 6  
2: 1, 3, 4, 5  
3: 0, 1, 4, 5  
4: 2, 3, 6  
5: 1, 2, 3, 6  
6: 1, 4, 5

a) QUEUE: 0

Print:

P: 0

b) Q: 0 1 3

P: 0 1

c) Q: 0 1 3 5 2

P: 0 1 3

d) Q: 0 1 3 5 2 4

P: 0 1 3 5

e) Q: 0 1 3 5 2 4

P: 0 1 3 5 2

f) Q: 0 1 3 5 2 4

P: 0 1 3 5 2 4

g) Q: 0 1 3 5 2 4 6

P: 0 1 3 5 2 4 6

h) Q: 0 1 3 5 2 4 6

P: 0 1 3 5 2 4 6 (Am.)



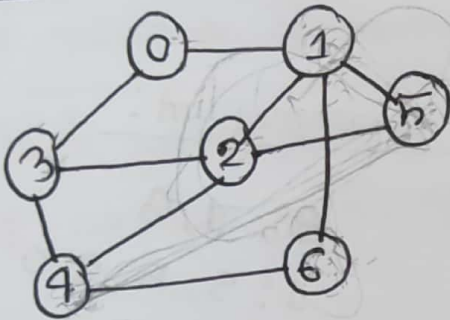


## Depth-First Traversal:

- Continue until all vertices reachable from the original source are discovered.

- It's backtrack the stack to traverse all vertices.

### Example:



a) Stack: 0

b) Stack: 1 3

c) S: 1 4 2

d) S: 1 4 5

e) S: 1 4

f) S: 1 6

g) S: 1

h) S:  $\emptyset$

P: 0

P: 0 3

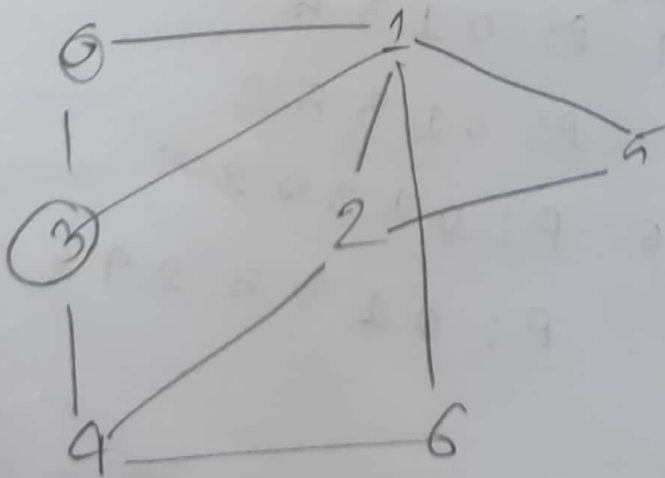
P: 0 3 2

P: 0 3 2 5

P: 0 3 2 5 4

P: 0 3 2 5 4 6

P: 0 3 2 5 4 1



## Differences between BFS and DFS:

| BFS                                                          | DFS                                                           |
|--------------------------------------------------------------|---------------------------------------------------------------|
| ① BFS stands for Breadth First Search                        | ① DFS stands for Depth First Search                           |
| ② BFS uses Queue Data structure.                             | ② DFS uses Stack Data structure.                              |
| ③ BFS builds the tree level by level                         | ③ DFS builds the tree sub-tree by sub-tree.                   |
| ④ It works on the concept of FIFO                            | ④ It works on the concept of LIFO                             |
| ⑤ In BFS, there is no concept of backtracking.               | ⑤ <del>DFS</del> DFS algorithm uses the idea of backtracking. |
| ⑥ <u>Application</u> : bipartite graphs, shortest path, etc. | ⑥ <u>App</u> : acyclic graphs, topological order etc.         |
| ⑦ BFS is slow compared to DFS                                | ⑦ DFS is fast as compared to BFS                              |
| ⑧ BFS requires more memory.                                  | ⑧ DFS requires less memory.                                   |





## Differences between Tree & graph:

| Graph                                                    | Tree                                                |
|----------------------------------------------------------|-----------------------------------------------------|
| ① It is a non-linear data structure.                     | ① It is also a non-linear data structure.           |
| ② There is no unique node.                               | ② There is a unique node which is known as root.    |
| ③ Graphs can form cycle.                                 | ③ Tree cannot form cycle.                           |
| ④ They can be directed or undirected.                    | ④ They are always directed.                         |
| ⑤ For finding shortest path in networking graph is used. | ⑤ For game trees, decision trees, the tree is used. |

Tree: A tree is an undirected graph which is connected and acyclic.

### Spanning Trees:

A spanning tree is an undirected graph  $G(V, E)$  is a subset of edges  $T \subseteq E$  that are acyclic and connect all vertices in  $V$ .

Spanning Tree must consists of exactly  $n-1$  edges, where vertex number is  $n$ .



- How many ways to take a spanning tree from a complete graph?

-  $n(n-2)$  where  $n$  = number of vertices

### Minimum spanning tree:

The minimum spanning tree of a network is a spanning tree in which the sum of its edge weights is guaranteed to be minimal.

- It is possible to have more than one minimum spanning tree but the sum of its edge weight is guaranteed to be minimal.

### Applications of MST:

- Network designing
- Transportation planning
- Circuit layout

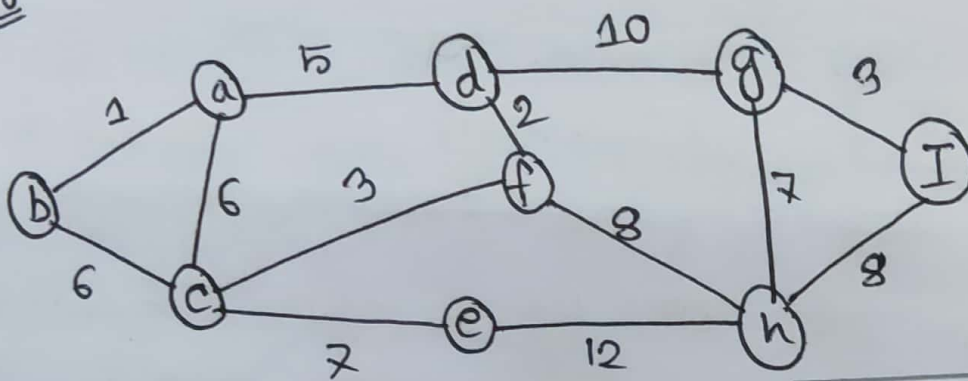
Two ways To solve MST:

MST — [ Prim's Algorithm  
Kruskal's Algorithm

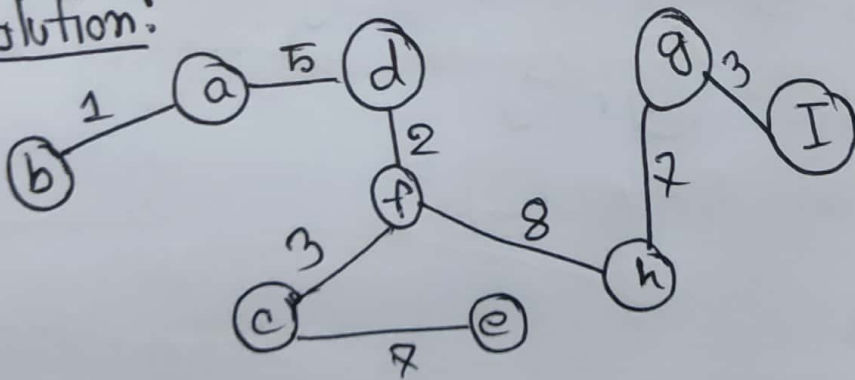
### Prim's Algorithm:

- Using greedy approach it is used to find minimum cost.

Example:



Solution:



Rules:

- Remove all parallel edge and loop edge

- एककाल node को source node से जोड़ें।

- Cycle को हटा दें।



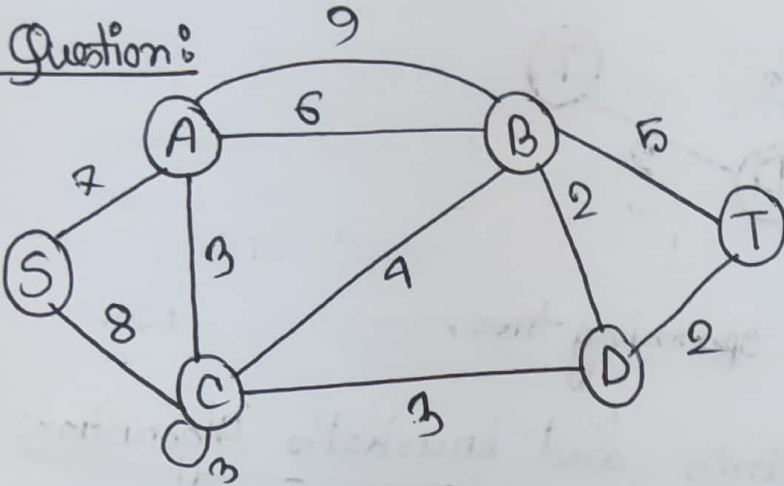


એમાં, (b) ને source node રીતે નિર્ણયિત, b ને edge ને  
 મળે minimal રીતે 1 તરીકે b to a edge  
 નિર્ણયિત, આ રીતે આ 6, એમાં (a) ને  
 minimal edge રીતે 5, આ રીતે આ 6, એમાં  
 મળે minimum 5, તરીકે (a) to (d). વાકી રીતે  
 આ 6, 6, (d) ને related edges રીતે 10, 2,  
 એમાં, 10, 2, 6, 6 ને minimum 2, તરીકે (d) to (f).  
 રીતે, 10, 6, 6, (f) ને adjacency edge 3, 8, 3, 8, 10,  
 6, 6, જે મળે 3 min. તરીકે (f) to (e). રીતે, 8, 10,  
 6, 6. (e) ને adjacency 6, 6, 7, 10, 6, 8, 6, જે મળે  
 6 min તરીકે 6, 6 જે ફોર cycle રીતે માલ તરીકે  
 7 નિર્ણયિત રીતે (e) to (c), રીતે 10, 8, (c) ને  
 adjacency 12, 10, 8, 12 ને min 8, તરીકે (c) to  
 (h), રીતે 12, 10, (h) ને adjacency 7, 8, 7, 8, 8 જે  
 min 7, તરીકે (h) to (g). (g) ને adjacency 3,  
 8, 3 જે min 3, તરીકે (g) to (i).

# Kruskal's Algorithm:

This is a greedy algorithm.

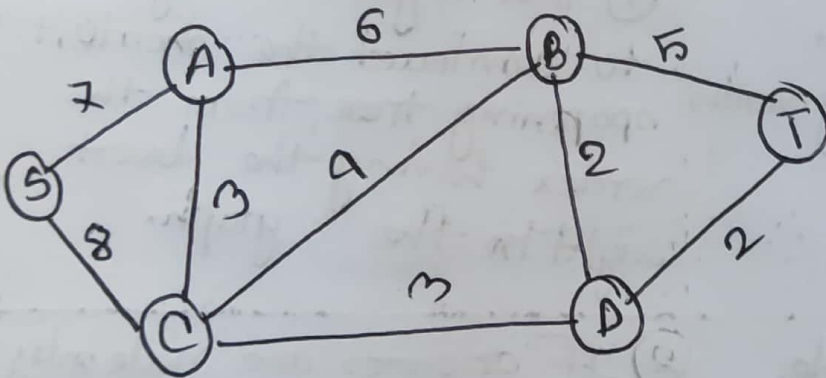
Question:



Solution:

- Remove all parallel edge and loop edge
- sort the edges into ascending order by weight
- No cycle
- ~~Connect~~ Set-0  
शून्य connect  
नया NA,

Step-1: Removing all parallel and loop edge



Step-2:

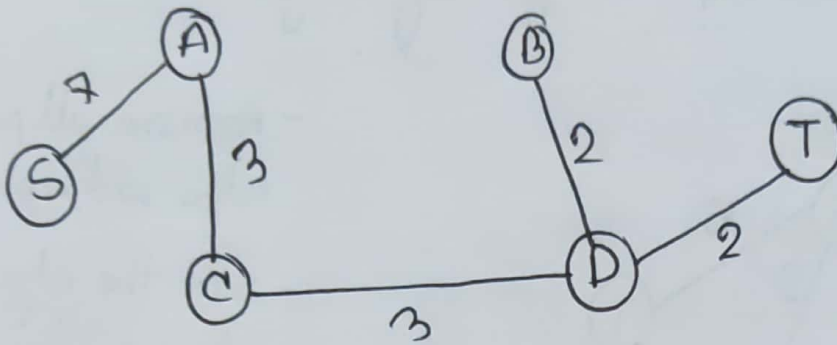
Sort the edges into ascending order.

|      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|
| B, D | D, T | A, C | C, D | B, C | B, T | A, B | A, S | S, C |
| 2    | 2    | 3    | 3    | 4    | 5    | 6    | 7    | 8    |



Step-3:

Sorting sequence ① different set - ② Add



It is the minimal spanning tree.

□ Differences between Prim's and Kruskal's Algorithms

| Prim's Algorithm                                                                              | Kruskal's Algorithm                                                                                                    |
|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| ① This Algorithm begins to construct the shortest spanning tree from any vertex in the graph. | ① This algorithm begins to construct the shortest spanning tree from the vertex having the lowest weight in the graph. |
| ② It traverses one node more than one time.                                                   | ② It crosses one node only one time.                                                                                   |
| ③ Time complexity: $O(V^2)$                                                                   | ③ Time complexity: $O(E \log V)$                                                                                       |
| ④ In this algorithm, all the elements must be connected.                                      | ④ In this algorithm, may have disconnected graph.                                                                      |
| ⑤ When it comes to dense graphs, the Prim's algorithm runs faster.                            | ⑤ When it comes to sparse graphs, Kruskal's algorithm runs faster.                                                     |



⑥ It prefers list data structure

⑤ It prefers the heap data structure.

